

P2201R1: Mixed string literal concatenation

Introduction

String concatenation involving *string-literals* with *encoding-prefixes* mixing L^{""}, u8^{""}, u^{""}, and U^{""} is currently conditionally-supported with implementation-defined behavior (5.13.5 [lex.string] paragraph 11).

None of icc, gcc, clang, MSVC supports such mixed concatenations; all issue an error: <https://compiler-explorer.com/z/4NDo-4>. Test code:

```
void f() {  
    { auto a = L"" u""; }  
    { auto a = L"" u8""; }  
    { auto a = L"" U""; }  
  
    { auto a = u8"" L""; }  
    { auto a = u8"" u""; }  
    { auto a = u8"" U""; }  
  
    { auto a = u"" L""; }  
    { auto a = u"" u8""; }  
    { auto a = u"" U""; }  
  
    { auto a = U"" L""; }  
    { auto a = U"" u""; }  
    { auto a = U"" u8""; }  
}
```

SDCC, the Small Device C Compiler, does support such mixed concatenations, apparently taking the first *encoding-prefix*. The sentiment was expressed that the feature is not actually used much, if at all: [WG14 e-mail](#)

No meaningful use-case for such mixed concatenations is known.

This paper makes such mixed concatenations ill-formed.

History

The history was kindly provided by Alisdair Meredith, although all errors should be blamed on the author.

Concatenating narrow and wide string literals was made defined behavior for C++11 by Clark Nelson's paper synchronizing with the C99 preprocessor: [N1653](#).

The conditionally supported implementation-defined behavior for concatenating unicode and wide string literals was a feature of the original proposal for unicode character types: [N2249](#).

The final rule to make u8 literals ill-formed when attempting to concatenate with a wide string literal was in the original paper proposing u8 literals: [N2442](#)

Changes in R1 vs. R0

- Approved by SG16, EWG, and CWG.
- Added Annex C entries.

Wording changes

Change in 5.13.5 [lex.string] paragraph 11:

In translation phase 6 (5.2 [lex.phases]), adjacent *string-literals* are concatenated. If both *string-literals* have the same *encoding-prefix*, the resulting concatenated *string-literal* has that *encoding-prefix*. If one *string-literal* has no *encoding-prefix*, it is treated as a *string-literal* of the same *encoding-prefix* as the other operand. ~~If a UTF-8 string literal token is adjacent to a wide string literal token, the program is ill-formed.~~ Any other concatenations are ~~conditionally-supported-with-implementation-defined-behavior~~ **ill-formed**. [Note: This concatenation is an interpretation, not a conversion. Because the interpretation happens in translation phase 6 (after each character from a *string-literal* has been translated into a value from the appropriate character set), a *string-literal*'s initial rawness has no effect on the interpretation or well-formedness of the concatenation. — end note] Table 11 has some examples of valid concatenations.

(Table 11)

Characters in concatenated strings are kept distinct. [Example:

```
"\xA" "B"
```

contains the two characters 'xA' and 'B' after concatenation (and not the single hexadecimal character 'xAB'). — end example]

[Insert a new subclause C.1 "C++ and ISO C++ 2020":](#)

Affected subclause: 5.13.5 [lex.string]

Change: Concatenated *string-literals* can no longer have conflicting *encoding-prefixes*.

Rationale: Removal of unimplemented conditionally-supported feature.

Effect on original feature: Concatenation of *string-literals* with different *encoding-prefixes* is now ill-formed. [Example:

```
auto c = L"a" U"b"; // was conditionally-supported; now ill-formed
```

-- end example]

[Add to C.5.1 \[diff.lex\]:](#)

Affected subclause: 5.13.5 [lex.string]

Change: Concatenated *string-literals* can no longer have conflicting *encoding-prefixes*.

Rationale: Removal of non-portable feature.

Effect on original feature: Concatenation of *string-literals* with different *encoding-prefixes* is now ill-formed.

Difficulty of converting: Syntactic transformation.

How widely used: Seldom.